



UNIVERSIDAD PRIVADA DE TACNA

Proyecto: Sistema Verificador de Cumplimiento OWASP

Curso: Calidad Y Pruebas De Software

Docente: Mag. Ing. Patrick Cuadros Quiroga

Integrantes:

Concha Llaca, Gerardo Alejandro

(2017057849)

Andía Navarro, Diego Fabrizio

(2022073906)

Tacna – Perú

2026



CONTROL DE VERSIONES					
Versión	Hecha por	Revisada por	Aprobada por	Fecha	Motivo
07	Andia Navarro, Diego Fabrizio Concha Llaca, Gerardo Alejandro	Mag. Ing. Patrick Cuadros Quiroga	Mag. Ing. Patrick Cuadros Quiroga	27/06/26	

Sistema Verificador de Cumplimiento OWASP

Documento de Visión
Versión {1.0}

Índice

1. Error! Bookmark not defined.1.1Error! defined.1.2Error!	Bookmark	not
	Bookmark	not



defined.1.3Error!	Bookmark	not
defined.1.4Error!	Bookmark	not
defined.1.5Error!	Bookmark	not
defined.2.Error!	Bookmark	not
defined.2.1Error!	Bookmark	not
defined.2.2Error!	Bookmark	not
defined.3.Error!	Bookmark	not
defined.3.1Error!	Bookmark	not
defined.3.2Error!	Bookmark	not
defined.3.3Error!	Bookmark	not
defined.3.4Error!	Bookmark	not
defined.3.5Error!	Bookmark	not
defined.3.6Error!	Bookmark	not
defined.4.Error!	Bookmark	not
defined.4.1Error!	Bookmark	not
defined.4.2Error!	Bookmark	not
defined.4.3Error!	Bookmark	not
defined.4.4Error!	Bookmark	not
defined.5.Error!	Bookmark	not
defined.6.Error!	Bookmark	not
defined.7.Error!	Bookmark	not
defined.8.Error!	Bookmark	not
defined.9.Error!	Bookmark	not
defined.CONCLUSIONES		23
RECOMENDACIONES		24

1. Introducción

1.1 Propósito

El propósito de este proyecto es desarrollar un **Ecosistema Verificador de Cumplimiento OWASP** (servicios web y extensión para el IDE VS Code) que optimice y facilite el proceso de aseguramiento de la calidad de software y desarrollo seguro (*DevSecOps*) para desarrolladores independientes y empresas de software. El sistema está diseñado para proporcionar diagnósticos y retroalimentación inmediata sobre vulnerabilidades de seguridad (OWASP Top 10 y dependencias CVEs) directamente en el editor del programador y en un dashboard web centralizado para auditores.

1.2 Alcance



El proyecto tiene como alcance desarrollar e implementar un sistema multiplataforma diseñado para escanear de forma estática código fuente local, URLs públicas y repositorios de GitHub, teniendo como piloto de despliegue los servidores en la nube de **Securify Software Solutions S.A.C.** El sistema permite al equipo de TI automatizar la auditoría de seguridad, centralizar logs de accesos de red y descargar reportes interactivos en PDF, mientras que los desarrolladores acceden a sugerencias de remediación y marcas diagnósticas en VS Code.

1.3 Definiciones, Siglas y Abreviaturas

- **SAST (Static Application Security Testing):** Pruebas de seguridad que analizan código fuente estáticamente sin ejecutar la aplicación.
- **OWASP (Open Web Application Security Project):** Estándar internacional y consorcio de referencia para vulnerabilidades web comunes.
- **CVE (Common Vulnerabilities and Exposures):** Diccionario estandarizado de vulnerabilidades de dependencias públicas.
- **IDE (Integrated Development Environment):** Entorno de desarrollo integrado del desarrollador (ej: Visual Studio Code).
- **API REST:** Interfaz basada en HTTP que permite comunicar el IDE con el servidor de análisis.
- **SSRF (Server-Side Request Forgery):** Vulnerabilidad explotada en red que es bloqueada en la verificación de URLs del sistema.

1.4 Referencias

- OWASP Foundation. (2021). *OWASP Top 10:2021 The Core 10 Security Risks*. OWASP Project.



- Microsoft. (2025). *Visual Studio Code Extension API Documentation*. Microsoft Learn.
- FastAPI Dev Team. (2025). *FastAPI framework reference documentation*. Python Foundation.

1.5 Visión General

El proyecto consiste en un sistema de análisis estático modular en capas. El núcleo corre bajo Python/FastAPI conectándose con persistencia dual SQLite3 (local) y MySQL (producción). El sistema interactúa de forma asíncrona con la API de GitHub para descargar y reportar issues de error, mientras que el frontend del IDE (JavaScript) subraya líneas vulnerables de código al guardar el archivo y expone un dashboard visual glassmorphic premium de cumplimiento.

2. Posicionamiento

2.1 Oportunidad de negocio

El 85% de las vulnerabilidades en software comercial se introducen en las fases tempranas de la codificación. Las herramientas de seguridad tradicionales (SAST comerciales como Checkmarx o Veracode) son excesivamente costosas y difíciles de configurar para equipos pequeños o desarrolladores freelance. Nuestro sistema representa una oportunidad de mercado de bajo costo (Freemium/B2B) al ofrecer auditoría en tiempo real directamente en el IDE de VS Code y reportes interactivos ágiles listos para imprimir, acelerando la entrega de código seguro a producción sin ralentizar las estaciones de trabajo locales.



2.2 Definición del problema

Elemento	Descripción
El problema de	Falta de retroalimentación inmediata sobre vulnerabilidades de seguridad (OWASP) y dependencias desactualizadas (CVEs) durante la fase de desarrollo.
Afecta a	Desarrolladores independientes (freelance), analistas de QA y consultoras de software como Securify Software Solutions S.A.C.
Cuyo impacto es	Exposición involuntaria de secretos (API keys), inyección de código inseguro, multas legales por brechas de datos y el costo de corregir errores en producción.
Una solución exitosa sería	Un ecosistema integrado ligero que analice código y dependencias al guardar archivos en el editor, exponga un dashboard de cumplimiento con sugerencias de remediación y guarde registros históricos.

3. Descripción de los interesados y usuarios

3.1 Resumen de los interesados

- **Desarrolladores (Freelance / In-house):** Interesados en escribir código seguro de forma rápida y sin interrupciones.
- **Líderes de TI / Auditores de Seguridad:** Responsables de certificar que el software no contenga vulnerabilidades antes de pasar a producción.
- **Docentes y Estudiantes de Ingeniería (UPT):** Interesados en herramientas de calidad que sirvan de laboratorio práctico sobre OWASP y DevSecOps.

3.2 Resumen de los usuarios

Nombre	Descripción	Rol en el Sistema
Usuario Desarrollador	Escribe código en su estación local e interactúa con los subrayados y el dashboard en su IDE.	Consumidor de diagnósticos.



Administrador / Auditor	Administra las configuraciones, gestiona usuarios de base de datos, revoca tokens de API y consulta logs de red.	Supervisor y Auditor.
API Cliente (CI/CD)	Pipelines y scripts de integración continua que consumen el backend de forma automatizada.	Integración automática.

3.3 Entorno de usuario

El usuario interactúa en un entorno híbrido:

1. **IDE Local:** La extensión de VS Code corre de fondo en la máquina del desarrollador, activándose en cada guardado de archivos.
2. **Dashboard Web Administrativo:** Interfaz HTML/Jinja2 responsive accesible desde cualquier navegador moderno en computadores de escritorio o móviles para supervisar logs y usuarios.

3.4 Perfiles de los interesados y usuarios

- **Desarrolladores:** Poseen altos conocimientos de programación pero baja familiaridad con las firmas Regex de OWASP. Prefieren interfaces visuales en el editor con Quick Fix y ayudas visuales claras.



- **Audidores de TI:** Tienen conocimientos en análisis de riesgos de red y seguridad.

Buscan reportes consolidados, métricas de cumplimiento (calificación 0-100) y logs detallados de IP y rutas solicitadas.

3.5 Necesidades de los interesados y usuarios

CÓDIGO	NECESIDADES	CÓDIGO REQ	REQUERIMIENTO FUNCIONAL (RF)	PRIORIDAD	IMPORTANCIA
N-01	Analizar código y URLs de forma ágil.	RF01	Escanear objetivos (código, archivos, URLs y repositorios GitHub).	Alta	Crítica
N-02	Detectar librerías externas vulnerables.	RF02	Identificar dependencias inseguras mediante base de datos CVE.	Alta	Alta
N-03	Comunicarse de forma segura con la API.	RF03	Generar y validar tokens de API (X-API-Key).	Alta	Crítica



N-04	Controlar accesos administrativos.	RF04	Gestionar sesiones administrativas (login/logout).	Alta	Alta
N-05	Administrar usuarios de la plataforma.	RF05	Gestionar usuarios y roles (crear, editar, eliminar).	Alta	Media
N-06	Exportar reportes físicos a clientes.	RF06	Exportar reportes detallados en formato PDF y JSON.	Alta	Alta
N-07	Diagnóstico inmediato en el editor.	RF07	Visualizar marcas de diagnóstico y problemas en el IDE.	Alta	Crítica
N-08	Guía de remediación del código.	RF08	Guiar remediación y Quick Fixes interactivos en VS Code.	Media	Alta
N-09	Trazas y auditoría de accesos de red.	RF09	Monitorear accesos de red y logs de auditoría (IP, ruta, usuario).	Media	Media

4. Vista General del Producto

4.1 Perspectiva del producto

El producto actúa como un intermediario de seguridad (*Security Gate*) entre la escritura de código en la máquina local y el despliegue del proyecto. El backend FastAPI centraliza las reglas de evaluación, permitiendo que múltiples clientes (IDE, CLI, navegador) consuman los mismos servicios y almacenen sus reportes históricos en una base de datos relacional MySQL/SQLite3 unificada.



4.2 Resumen de capacidades

- **Auditoría en Caliente:** Análisis al guardar en el editor VS Code sin necesidad de subir archivos manuales.
- **Cálculo de Calificación Inteligente (RB03):** Puntuación base 100 con penalizaciones proporcionales a la severidad del fallo (Alta: -30, Media: -15, Baja: -5).
- **Integración y tickets automáticos:** Conector directo con la API de GitHub para abrir incidencias (issues) de seguridad con un solo clic.

4.3 Suposiciones y dependencias

- Se asume que la extensión requiere conexión HTTP/HTTPS con el backend FastAPI (local o desplegado en Debian VM) para ejecutar los escaneos de código.
- El escaneo de dependencias asume que los proyectos incluyen archivos estándar de configuración (`requirements.txt`).

4.4 Licenciamiento e instalación

- El backend se instala localmente o en un servidor Debian mediante Git y dependencias de `requirements.txt`. El licenciamiento de desarrollo es de código abierto bajo Licencia MIT.

5. Características del producto

- **Análisis Regex modular:** Motor de firmas extensible cargado desde diccionarios de variables.



- **Identificación CVE integrada:** Diccionario de dependencias preestablecidas de seguridad.
 - **Auditoría de cabeceras HTTP:** Verificación CSP, HSTS y directivas X-Content de URLs.
 - **Integración GitHub:** Sincronización y empuje asíncrono de tickets de vulnerabilidad.
 - **Extensión de VS Code:** Diagnósticos del IDE, resaltado de líneas, barra de estado y panel administrativo glassmorphic.
-

6. Restricciones

- El backend requiere Python 3.11 o superior.
 - El cliente IDE requiere tener instalado Microsoft VS Code v1.85+.
 - Por motivos de seguridad (RB02), el escáner de URLs bloquea accesos directos a localhost o interfaces de red privadas para mitigar ataques SSRF.
-

7. Rangos de calidad

Funcionalidad

- **Rango Bajo:** El sistema presenta fallas en los escaneos, no resalta las líneas en el editor o genera calificaciones erróneas.
- **Rango Alto:** Escaneo preciso, reporte correcto de hallazgos bajo regla de penalización y mapeo de diagnósticos sin fallos en VS Code.

Rendimiento



- **Rango Bajo:** Los análisis toman más de 15 segundos en archivos locales de código, congelando la escritura en el editor.
 - **Rango Alto:** Tiempo de escaneo de código local menor a 5 segundos y respuesta de la API HTTP a la extensión en menos de 150ms.
-

8. Precedencia y Prioridad

Alta Prioridad

- **RF01/RF07 (Escaneo y Diagnósticos en IDE):** El núcleo del sistema es detectar fallos e informarle al programador en tiempo real.
- **RF02 (Análisis de dependencias CVE):** Identificación automatizada de paquetes inseguros.
- **RF04 (Sesiones administrativas):** Login y logout de control administrativo.

Media Prioridad

- **RF06 (Exportación PDF/JSON):** Generación de reportes limpios para entregas físicas de proyectos.
 - **RF09 (Monitorear logs de accesos):** Histórico de IP y rutas para auditoría.
-

9. Otros requerimientos del producto

a) Estándares legales



- **Ley de Protección de Datos Personales (Ley No. 29733):** Asegurar que las trazas de auditoría y base de datos relacional traten la información de accesos con confidencialidad.
- **Licencia de Software Libre MIT:** Garantizar la libre redistribución y el uso sin restricciones de propiedad intelectual de las librerías dependientes.

b) Estándares de comunicación

- **HTTPS/HTTP:** Cifrado obligatorio para el tráfico de red de fragmentos de código.
- **REST API:** Arquitectura de comunicación modular y ligera entre VS Code/CLI y el backend FastAPI.
- **JSON:** Serialización ligera de metadatos de escaneo.

c) Estándares de cumplimiento de la plataforma

- El backend opera bajo cualquier sistema operativo compatible con Python (Windows, macOS, y Linux Debian). El IDE cliente opera sobre VS Code de Microsoft.
- Persistencia dual: SQLite3 nativo local sin requerir motor de base de datos pesado y MySQL relacional estándar en Debian.

d) Estándares de calidad y seguridad

- **ISO/IEC 27001:** Adopción de controles básicos de seguridad de la información.
- **OWASP Top 10 Compliance:** Cobertura de las firmas estáticas de código y cabeceras de red para los estándares internacionales de mitigación web.



Conclusiones y Recomendaciones

Conclusiones

- El ecosistema optimiza drásticamente los tiempos de aseguramiento de calidad al integrar las comprobaciones directamente en el flujo de guardado de archivos en el editor.
- La interfaz glassmorphic premium de la extensión y el dashboard centralizado proveen una experiencia de usuario sumamente visual que promueve las buenas prácticas de desarrollo.
- La persistencia liviana y desacoplada permite conmutar la base de datos de SQLite3 a MySQL en producción sin añadir dependencias pesadas en el servidor.

Recomendaciones

- Mantener actualizado el diccionario de CVEs y la base de firmas de OWASP en `cve_analyzer.py` y `scanner.py` para responder a nuevos riesgos.
- Integrar el motor de comandos de consola del CLI (`cli.py`) en flujos de integración continua (CI/CD) para automatizar el análisis antes de cada despliegue.